REFERENCE • UNDERSTAND THE WHOLE SYSTEM

# System Study Guide

How the Ynclino Apartment Management System is built and how it flows

This guide explains what the system is, how the code is organised, the data it stores, and the step-by-step flows behind each feature. Read it once end-to-end to get the big picture, then use the section headings to jump back to any part you need.

## 1. What the system is

Ynclino AMS is a web application for running a small apartment building. An administrator manages units, tenants, billing, maintenance, lost & found, and unit-transfer requests; a tenant logs in to see their own unit, bills, and requests. Everything runs in the browser.

### Technology stack

| Layer | Technology |
|---|---|
| Framework | ASP.NET Core MVC 8 (.NET 8) |
| Language | C# |
| Database | MySQL / MariaDB |
| Data access | Entity Framework Core 8 with the Pomelo MySQL provider |
| UI | Razor views (.cshtml) styled with Bootstrap 5 |
| Auth | Cookie authentication with role claims (Admin / Tenant) |

## 2. The two roles

Every logged-in user is either an Admin or a Tenant. The role is stored on their account and decides what they can see and do. Controllers and actions are protected with [Authorize] and [Authorize(Roles = "...")] attributes.

- Admin — full management: create/edit units, register tenants, issue bills, approve transfers and claims, handle maintenance, manage user accounts.
- Tenant — self-service: view their unit and bills, submit maintenance and lost & found reports, request a unit transfer, edit their profile.

# 3. How an MVC request flows

MVC stands for Model–View–Controller. Each browser request travels the same path:

1. Routing sends the URL (e.g. /Billing/Index) to a Controller action method.
2. The Controller asks the DbContext (EF Core) for data from MySQL.
3. EF Core returns Model objects (rows mapped to C# classes).
4. The Controller passes those models to a View (.cshtml).
5. The View renders HTML, which is sent back to the browser.

```
Browser  ->  Controller  ->  DbContext (EF Core)  ->  MySQL
Browser  <-    View      <----------  Model  <---------
```

## Where each piece lives

| Folder | What it holds |
|---|---|
| Controllers/ | Request handlers — one per module (Units, Tenants, Billing, ...) |
| Models/ | Data classes (tblUnit, tblTenant, ...) mapped to database tables |
| Views/ | Razor pages that render HTML, grouped by controller |
| Data/ | ApplicationDbContext — the EF Core gateway to the database |
| Helpers/ | Reusable logic: passwords, image uploads, notifications |
| ViewComponents/ | Small reusable UI widgets (e.g. notification badges) |

# 4. The data model

Each table is a C# class in Models/ (the tbl-prefixed classes). These are the core tables and how they relate to each other.

| Table | Represents | Key links |
|---|---|---|
| tblUser | A login account + role | 1 tenant may link to 1 user |
| tblUnit | An apartment unit | has many tenants, bills |
| tblTenant | A person renting a unit | belongs to a unit & a user |
| tblBilling | A monthly bill | belongs to a tenant/unit |
| tblMaintenanceRequest | A repair request | raised by a tenant |
| tblLostFoundItem | A lost or found item | reported by a user |
| tblClaimRequest | A claim on a found item | links item + claimant |
| tblUnitTransferRequest | A move/transfer request | tenant + from/to unit |
| tblNotification | An in-app alert | targets users per module |

The database and all these tables are created automatically on first run (see the Database Connection Guide). You never write CREATE TABLE by hand.

## 5. Signing in

AccountController handles login. On submit it looks up the username, verifies the password with the hashing helper, and if valid issues an authentication cookie carrying the user's id and role. That cookie is what every later [Authorize] check reads.

- Default admin (seeded on first run): admin / Admin@123.
- Sample tenants (after Load Sample Data) use the password Tenant@123.
- Logout clears the cookie; Change Password updates the stored hash.

## 6. The tenant rent journey (start to finish)

This is the main end-to-end flow — how a person goes from nothing to a paying, settled tenant.

1. Admin adds the unit. Under Units, the admin creates the apartment with its rent, deposit, advance, and capacity. New units start as Vacant.
2. Admin registers the tenant. Under Tenants, the admin records the person's details and assigns them to a vacant unit, optionally creating a login account for them.
3. The unit fills up. Once assigned, the unit's status reflects occupancy (Occupied when full), so it no longer shows as available.
4. Admin issues bills. Under Billing, the admin creates a monthly bill for the tenant (rent, plus deposit/advance on move-in). Each bill has an amount due and a due date.
5. Tenant logs in and reviews. The tenant sees their unit, their bills, and the balance on their dashboard and Billing page.
6. Payments are recorded. As the tenant pays, the admin records the amount paid; the bill's status is recalculated automatically (see section 7).
7. Life happens. The tenant can submit maintenance requests, report lost & found items, or request to transfer to another vacant unit — each handled in its own module below.
8. Move-out. When the tenancy ends, the admin deactivates the tenant; the unit returns to Vacant and can be rented again.

## 7. Billing — automatic status

You never pick a bill's status by hand. BillingController derives it from the amount due, the amount paid, and the due date, so it is always consistent.

| Status | Meaning |
|---|---|
| Paid | Paid in full. |
| Partial | Part paid, a balance remains, not yet past due. |
| Unpaid | Nothing paid yet, not yet past due. |
| Late | Past the due date and not paid in full. |

## 8. Maintenance requests

A tenant reports a problem; the admin works it to completion.

1. Tenant submits a request describing the issue and choosing a priority (Minor, Moderate, Major, or Urgent). A photo can be attached.
2. The admin sees it, updates the status as work progresses, and closes it when done.
3. The tenant follows the status from their side the whole time.

## 9. Lost & Found and ownership claims

Two item categories exist: Lost (someone lost it) and Found (someone handed it in).

1. A user reports an item, optionally with a photo and the location.
2. For a Found item, another user can file a Claim, describing proof of ownership (and attaching a proof photo).
3. The admin reviews each claim and Approves or Rejects it, optionally leaving a note.
4. Once settled, the admin marks the item Resolved.

## 10. Unit transfer requests

A tenant who wants to move to a different vacant unit files a request instead of moving themselves.

1. Tenant opens Create under Transfers, picks a vacant unit, and (optionally) gives a reason.
2. The request is Pending and the admins are notified.
3. The admin Approves (which moves the tenant to the new unit and frees the old one) or Rejects it. The tenant may Cancel their own request while it is still pending.
4. Reviewed requests move into an archive so the active list stays clean.

## 11. Notifications

The system keeps people informed with in-app notifications instead of email.

- When something needs attention (a new request, a decision), NotificationHelper creates a notification aimed at the right people — e.g. all admins, or one tenant.
- Each module shows an unread count badge; opening the related item marks it read, tracked per user, so your unread state is your own.
- ViewComponents (NavAlert, ModuleNotifications) render these badges and lists on the page.

## 12. How the database is created and kept in sync

At startup Program.cs calls EnsureCreated(): if the database is missing it is built from the current models; if a leftover database no longer matches the code, the app drops and rebuilds it. It then seeds the default admin. This is why each branch must use its own database name.

> **Study tip**
> To see any flow for real: run the app, log in as admin, click Load Sample Data on the Units page, then log in as a sample tenant (password Tenant@123) in a second browser. Doing an action as the tenant and watching it appear for the admin is the fastest way to understand the whole system.

## 13. Quick glossary

| Term | Plain meaning |
|---|---|
| MVC | Model-View-Controller — the app's three-part structure. |
| EF Core | The library that turns C# classes into database rows and back. |
| DbContext | The single object the code uses to read/write the database. |
| Model / tbl-class | A C# class that mirrors one database table. |
| Controller | Code that handles a request and decides what to show. |
| View (.cshtml) | The template that produces the HTML page. |
| Razor | The syntax that mixes C# into HTML in a view. |
| Claim / cookie | How the site remembers who you are and your role. |
| Seed | Insert starter data (like the default admin) automatically. |